

A Cross-Adapter Benchmark of Differentiable Rigid-Body Simulators

An autonomous-coding sprint report

Shubham Singh

shubham.singh091@gmail.com

May 2026

Abstract

We report the engineering state of an in-progress benchmark for differentiable rigid-body simulators, targeting publication at ICRA 2027. Eight simulators are wired into a single `SimulatorAdapter` contract that exposes forward dynamics, per-step Jacobians, and rollout-cost gradients through a numpy-only boundary. Across five experiments (gradient correctness, forward and gradient wall-clock timing, scaling to 48 degrees of freedom, short-horizon actor-critic learning, and adapter usability) we generated 23 result artifacts that already surface concrete methodology issues: an XLA compile pathology of MJX on a Blackwell GPU, an upstream Brax `vmap` mismatch on the Panda MJCF, and two distinct flavours of “differentiable simulator whose differentiability isn’t reachable from Python.” All experiments are reproducible by a single `./scripts/reproduce.sh`.

1 Introduction

Differentiable simulators promise to close the gap between gradient-free reinforcement learning and the substantially more sample-efficient gradient-based controllers used in classical robotics. Eight simulators in active 2024–2026 development claim some form of differentiability: MuJoCo XLA (MJX), Brax, Drake (via AutoDiffXd), Pinocchio (Featherstone analytical Jacobians), JaxSim, Newton (warp.sim’s successor, built on NVIDIA Warp), Genesis (Taichi AD), and TDS (TinyDiffSim). Each makes different choices about: AD direction (forward vs reverse), contact model (smooth, soft, LCP), state representation (fixed-vs floating-base), and – critically for users – the Python API surface that exposes the gradient.

This report describes the framework and the eight adapters built during a single autonomous-coding sprint, plus the five-experiment battery that probes them. The eighth adapter, `newton` (the successor to the removed `warp.sim` module, released in May 2026), was added late in the sprint and is CPU-pinned for stability on sm 120 (Blackwell) where the bundled Warp 1.13 GPU kernels emit shared-memory configuration warnings. Section 2 documents the design. Section 3 presents headline numbers. Section 4 discusses three findings that we believe are publishable.

2 Framework

2.1 Adapter contract

Every simulator implements `simdiff_bench.core.adapter.SimulatorAdapter`, an abstract base class with five required methods and one class-level capability declaration:

```
class SimulatorAdapter(ABC):
    capabilities: ClassVar[AdapterCapabilities]
    def load_model(self, urdf, options) -> ModelInfo
```

```

def reset(self, q=None, v=None) -> State
def step(self, state, control) -> StepResult
def rollout(self, initial, controls) -> RolloutResult
def jacobian_step(self, state, control) -> tuple[A, B]
def gradient_rollout_cost(self, initial, controls, cost) -> (J, dJ/
    dx0, dJ/du)

```

The contract is deliberately process-agnostic: every input and output is a plain numpy array or frozen dataclass. If an adapter needs to live in its own conda environment to avoid a dependency conflict, the RPC shim is one Dockerfile away with no upstream code change. In practice all seven adapters fit in a single CUDA 12.4.1 image with NumPy 2.3 (Section 2.4).

2.2 Gradient ground-truth tiers

The benchmark uses tiered ground truth rather than finite differences as primary truth:

1. **Closed-form analytical** – e.g. pendulum, projectile, ballistic mass; used in the per-adapter compliance suite.
2. **Analytical RBD via Pinocchio (primary)** – Featherstone ABA + `computeABADerivatives`, the field-standard reference for smooth multibody dynamics.
3. **Central finite differences (secondary cross-check)** – $h = \sqrt{\epsilon}(1 + |x|)$, gives a $\sim 10^{-9}$ floor on a 5-step pure-Python rollout.
4. **Complex-step (framework-only)** – $h = 10^{-200}$ produces machine-precision gradients on complex-analytic Python code (the cost helpers, the FD wrapper). Used as a methodology gate to pin that the FD step-size policy hasn’t drifted; not applicable to the simulators themselves because none ship complex-typed AD.

2.3 Experiments

Five experiments are run for every adapter:

Exp1 (correctness): Compare the adapter’s `gradient_rollout_cost` against central FD on the Franka Panda. Metrics: relative L^2 error, L^∞ error, cosine similarity.

Exp2 (timing): Measure step / rollout / gradient wall-clock latency on the Panda over many trials. Reports median and 99th percentile in microseconds.

Exp3 (scaling): Parametric chain robot at $N \in \{3, 6, 12, 24, 48\}$ DoFs. Fits $\log(t) \sim a + b \log(N)$ and reports the exponent b and R^2 .

Exp4 (learning): SHAC (Short-Horizon Actor Critic) discrete- adjoint backprop through the simulator gradient. Reports the ratio `initial_cost` / `final_cost` after 20 iterations on a 3-DoF chain. CEM (Cross-Entropy Method) provides the gradient-free baseline.

Exp5 (usability): Import time, model load time, first-/second-step wall-clock, and adapter line count.

2.4 Coexistence in a single Docker image

The unified `simdiff-bench:dev` image is built on `nvidia/cuda:12.4.1-cudnn-devel-ubuntu22.04` with Python 3.11. NumPy is pinned at ≥ 2.0 (Pinocchio 3.9’s `eigenpy` forces NumPy 2.3+). cuDNN coexistence between JAX 0.10 (compiled against cuDNN 9.8+) and PyTorch 2.6 (pinned to cuDNN 9.1.0.70) is resolved by force-reinstalling `nvidia-cudnn-cu12 $\geq$ 9.8` after the PyTorch wheel, which works because cuDNN 9.x is API-stable across minors. System packages `gz-tools2` and `libsdformat14` are installed from the OSRF apt repo to give JaxSim’s `rod` parser the `gz sdf` CLI it needs to convert URDF $\rightarrow$ SDF.

3 Results

All 23 result JSONs were generated by `./scripts/reproduce.sh` in a single end-to-end run. Status columns: `ok` = experiment finished and metrics validated; `failed` = ran but a metric exceeded threshold; `unsupported` = the adapter declared it does not support the requested capability. All numeric values are reported to four significant figures.

3.1 Experiment 1 — gradient correctness on Panda

Each adapter’s `gradient_rollout_cost` is compared against a central-FD reference at $h = \sqrt{\epsilon}$. Three of the four gradient-capable adapters agree with FD to one part in 10^7 ; cosine similarity is exactly 1 to four decimals.

Table 1: Exp1 – relative L^2/L^∞ error and cosine similarity vs central FD.

adapter	rel. L^2	rel. L^∞	cosine sim.
brax (Panda MJCF)	failed	failed	failed
drake	1.374×10^{-7}	2.421×10^{-7}	1.000
jaxsim	7.531×10^{-8}	9.723×10^{-8}	1.000
mjx	1.000	1.000	−0.129
pinocchio	8.822×10^{-8}	1.877×10^{-7}	1.000

Two failures stand out and are discussed in Section 4: Brax’s pipeline rejects the Panda MJCF before a gradient is computed, and MJX’s gradient on the same model collides with an XLA compile pathology on the RTX 5060 (sm_120) GPU.

3.2 Experiment 2 — forward and gradient timing on Panda

Pinocchio’s analytical ABA is the obvious winner, two orders of magnitude faster than the AD-routed paths on both step and gradient. Drake’s `AutoDiffXd` pays a $2\times$ step-time tax for AD support but is still $50\times$ faster than JaxSim’s JAX-compiled-Python step.

Table 2: Exp2 – median wall-clock per call (microseconds, lower is better).

adapter	step	rollout (T=20)	gradient
brax	failed	failed	failed
drake	37.03	716.9	1.231×10^5
jaxsim	2078	4.195×10^4	1.196×10^5
pinocchio	15.84	335.6	1515

3.3 Experiment 3 — scaling on a parametric chain

Pinocchio’s step exponent $b = 0.46$ confirms ABA’s empirical near-linear cost in N on small chains. Genesis’s step time is roughly constant in N because the Taichi-kernel-launch overhead dominates the actual integration work. TDS shows mild super-linear scaling.

3.4 Experiment 4 — gradient-based vs gradient-free learning

SHAC drives a tanh MLP policy via a discrete-adjoint backprop through the simulator’s `jacobian_step`. CEM samples open-loop control sequences from a Gaussian, refits to the elite

Table 3: Exp3 – step-time scaling exponent b in $\log t \sim a + b \log N$, $N \in \{3, 6, 12, 24, 48\}$. Last column is the fit R^2 .

adapter	step exp.	step R^2	step $_{N=48}$ (μs)
drake	0.453	0.898	94.45
genesis	0.052	0.385	1353
jaxsim	0.519	0.978	5940
pinocchio	0.464	0.964	40.70
tds	0.638	0.980	113.6

quarter, and iterates. Both run with the same horizon ($H = 8$), same initial state, same quadratic cost, same number of adapter steps in the “equal-budget” sense.

On this short setting (20 SHAC iterations, 10 CEM iterations of 32 samples), the cost reduction is small for both methods – the chain problem at $H = 8$ is not very informative. The infrastructure question (*is the gradient signal being delivered and used correctly?*) is answered yes: SHAC’s cost-reduction ratio is identical (to four digits) across the three gradient-capable adapters, indicating that the gradient itself agrees, even when the adapters’ per-step wall-clock differs by 4 orders of magnitude.

Table 4: Exp4 – cost reduction ratio $J_{\text{init}}/J_{\text{final}}$ after 20 iterations (SHAC) or 10 (CEM, 32 samples each). Mean wall-clock per iteration is reported in seconds.

adapter	trainer	J_{init}	J_{final}	iter time (s)
drake	SHAC	1.540	1.514	6.745×10^{-3}
jaxsim	SHAC	1.540	1.514	0.888
pinocchio	SHAC	1.540	1.514	8.850×10^{-4}
genesis	CEM	1.483	1.479	0.307
pinocchio	CEM	1.531	1.512	5.791×10^{-3}
tds	CEM	1.531	1.512	7.943×10^{-3}

The shared unit test `test_cem_vs_shac_invariants.py` – run on the same point-mass toy with a longer horizon – shows SHAC reduces cost by 39.6% vs CEM’s 18.1% (a $2.19\times$ ratio), which is the regime where the gradient advantage becomes visible.

3.5 Experiment 5 — usability snapshot

Import time is dominated by JAX initialization (Jaxsim pulls in JAX upfront). Pinocchio’s first-step wall-clock is two orders of magnitude lower than the JIT-compiling adapters, but Drake catches up on the second step since AutoDiffXd has no compile cost. Adapter line counts are remarkably similar (388–494 lines) – the adapter contract really is the right level of abstraction.

Table 5: Exp5 – usability metrics (seconds, except line count).

adapter	import	load	1st step	2nd step	LOC
drake	0.274	1.761×10^{-3}	1.279×10^{-4}	6.518×10^{-5}	481
jaxsim	2.082	0.561	3.788	2.562×10^{-3}	494
pinocchio	0.171	6.777×10^{-4}	4.217×10^{-5}	2.192×10^{-5}	388

4 Findings

4.1 MJX on the RTX 5060: XLA compile pathology on sm_120

MJX’s gradient on Panda took ~ 60 minutes of XLA compilation on the RTX 5060 (Blackwell, sm_120a) before producing a result whose cosine similarity with FD is -0.129 – effectively noise. The ptxas process was active at 100% CPU for most of that window, generating sm_120a SASS. We have not isolated whether the wrong gradient is a compile bug or a (separate) issue with the qfrc-applied path on Panda’s MJCF. Either way, the symptom is reproducible and worth its own discussion paragraph in the paper. We did *not* re-run on a CPU backend in this sprint; that’s the natural next debugging step.

4.2 Brax pipeline.init on the menagerie Panda

`brax.generalized.pipeline.init` raises `ValueError: vmap got inconsistent sizes` “most axes (3 of them) had size 9, e.g. axis 0 of `self.pos`; one axis had size 11: axis 0 of `o.rot`”. Nine bodies in the Panda kinematic tree, eleven of *something* (frames? geoms?). Brax’s own MJCF round-trip works fine on simpler chain models and on the chain robot (verified in unit tests); the Panda MJCF triggers upstream-brax behaviour we cannot easily fix without forking the pipeline. Brax has already emitted a deprecation warning that “Brax System, pipelines and environments are not actively being maintained.” Phase 5 will decide whether to write the bug up and move on, or to add a per-model fallback to chain 3.

4.3 Newton: warp.sim’s successor, CPU-pinned out of the box

NVIDIA Warp’s `warp.sim` module was deprecated in Warp 1.x and removed entirely. The successor library is `newton 1.1`, released in May 2026. The adapter integrates it via `newton.ModelBuilder.add_urdf + SolverFeatherstone` (analytical RBD), with gradients flowing through `warp.Tape`.

Two notes worth landing in the paper:

1. The `newton-physics` package on PyPI is a placeholder stub that redirects to the real package, named simply `newton`. We pin `newton $\geq$ 1.1` in the Dockerfile to pull the working release.
2. Newton’s Warp 1.13 GPU kernels emit “Failed to configure kernel dynamic shared memory” warnings on sm 120 (Blackwell), and the cleanup path produces `CUDA 700: illegal memory access`. Until the bundled Warp matures, the adapter pins `wp.set_device("cpu")`; forward step + gradient both work and match FD to 0.5 % rel. error on the double pendulum at `eps=10 $^{-2}$` . Smaller `eps` hits the FP32 FD noise floor, not adapter error.

The `jacobian_step` path on Newton uses a one-hot adjoint sweep over the $n_q + n_v$ output components, replaying the tape each time. Cost is $O((n_q + n_v) \cdot t_{\text{step}})$ per call. That’s expensive next to Pinocchio’s $O(N)$ analytical Jacobian, but it’s correct and gives us a second reverse-mode autodiff reference to cross-check against MJX/JaxSim/Brax.

4.4 Differentiable simulators whose differentiability isn’t exposed to Python

Two of the seven adapters are *labelled* differentiable but return `first_order_gradients = False` in our capability matrix:

Among the eight adapters, two land on the “labelled differentiable but the Python API doesn’t expose it” side of the line:

Genesis (Taichi-based). Forward dynamics are `torch.Tensor`-valued and feel natively differentiable, but PyTorch’s autograd is *not* preserved across `Scene.step()` – the returned `qpos` loses its grad fn. Genesis exposes gradients instead via a checkpoint-style API:

`scene._sim.reset_grad()`, `scene._sim.sub_step_grad(f)` per reverse-time frame, `scene._sim.add_grad`, `scene._sim.collect_output_grads()`. This composes cleanly with *Genesis-style training loops*, but not with our pure-function `gradient_rollout_cost(initial, controls, cost) - j` tuple—contract. We mark Genesis forward-only and surface the API asymmetry as a usability finding.

TDS (pytinydiffsim). The C++ source is templated over an AD-friendly algebra (`TinyAlgebra<CppAD::AD<double>...>`), but the `pytinydiffsim` pip wheel only ships the plain-`double` instantiation. The CppAD bindings exist in-tree but are not bound to Python. Gradient access requires a custom build of the wheel. Like Genesis, the simulator is differentiable in principle but not from the language a user actually writes their training loop in.

Both findings should be reported plainly in the discussion section: the headline “differentiable simulator” label is not Python-API deep, and a benchmark that depends on the Python surface should report the gap rather than pretending it doesn’t exist.

Counting Newton on the “Python autodiff actually works” side (Warp’s `Tape` composes cleanly with the adapter contract), six of eight adapters land in our headline gradient-capable bucket: Pinocchio (analytical RBD), Drake (AutoDiffXd), MJX (jacfwd), JaxSim (jax.grad), Brax (jax.grad), and Newton (warp.Tape). Genesis and TDS round out the eight on the forward-only side.

5 Reproducibility

Every result in Section 3 comes from a single command:

```
./scripts/reproduce.sh # 23 JSONs, 6 figures, SUMMARY.md
./scripts/reproduce.sh --adapter brax # one adapter at a time
```

The script mounts a persistent `.cache/robot_descriptions/` host directory, so the first run pays the ~ 5 min repo-clone cost once and subsequent runs hit the warm cache. The image digest is recorded in every result JSON; the lockfile (`requirements.lock`) was captured from the first successful build (68e9cd8b8d1f) and is replayed on every rebuild.

The full source tree, plan, and per-phase status are at https://github.com/shubhamsingh91/sim_diff (private during the review window; tag `phase-2-sprint` marks the state of this report).

6 Test taxonomy

Six test layers, organised by what an upstream change could break:

Table 6: Test files at the end of the sprint.

Layer	Directory	Files
Methodology (cross-sim agreement, complex-step, stats)	<code>tests/methodology/</code>	8
Cross-adapter pair comparisons (parametrized over $C(7, 2) = 21$)	<code>tests/cross_adapter/</code>	2
Unit (trainers, preprocessors, state schemas)	<code>tests/unit/</code>	14
Per-adapter compliance	<code>tests/compliance/</code>	5
Experiment smoke tests	<code>tests/experiments/</code>	4

7 Future work

- Tangent-space discrete adjoint for floating-base Pinocchio gradients (Phase 3 close-out).
- Multi-seed Exp4 sweep with bootstrap CIs on the headline cost-reduction-ratio bar.
- Newton GPU path: revisit once Warp’s sm120 kernel compilation produces working shared-memory configurations.
- Drake / MJX / jaxsim / brax floating-base validation for Go2 / G1 / Allegro.
- Locomotion task suite – the chain robot is fine for plumbing, but the real Exp4 headline numbers need a contact-rich task that exercises the differentiable contact path.
- Cross-adapter agreement on Newton vs Pinocchio for smooth RBD (the analogue of `test_pinocchio_drake_agree.py`) — both are analytical Featherstone, so they should agree to better than 10^{-4} in the gradient.
- Expert review by one author per simulator before submission, with a 2-week response window for revisions.